Research Journey: From Regret Replay to Learning-Progress Replay

This note records the current mathematical position of the project. The codebase started as a compact JAX implementation of unsupervised environment design (UED), with domain randomization, PLR/RPLR, ACCEL, and PAIRED as reference methods. The current research direction is narrower and more experimental: we are using the existing PLR machinery as a controlled substrate for asking which scalar score should cause a level to be replayed.

The central question is:

> Given a generated level, can we score it by the amount of useful learning it induces, rather than only by regret-like proxies?

In the current implementation, the candidate answers are:

1. classical PLR regret scores, `MaxMC` and `pvl`;
2. absolute raw policy-gradient magnitude, `abs_pg`;
3. PPO value-loss magnitude, `ppo_value_loss`;
4. normalized holdout learning progress, `s_in`.

The first two are inherited baselines. The latter three represent the present research frontier.

1 Shared Setting

Let x denote a level sampled from the level space $\mathcal{X}$. A policy $\pi_\theta(a \mid o)$ and value function $V_\theta(o)$ are trained with PPO on rollouts from levels. For a rollout of horizon T over N parallel levels, we write:

- $o_{t,n}$ for the observation at step t on level n;
- $a_{t,n}$ for the sampled action;
- $r_{t,n}$ for the reward;
- $d_{t,n}$ for the episode termination indicator;
- $v_{t,n} = V_\theta(o_{t,n})$;
- $A_{t,n}$ for the GAE advantage;
- $y_{t,n} = A_{t,n} + v_{t,n}$ for the value target.

The level sampler maintains a finite buffer

$$\mathcal{B} = (x_i, s_i, \tau_i)_{i=1}^{m}. \tag{1}$$

Here s_i is the current score of level x_i, and τ_i records when the level was last sampled. Replay weights combine score prioritization and staleness:

$$w_i = (1-\rho)\, w_i^{\text{score}} + \rho\, w_i^{\text{stale}}. \tag{2}$$

For rank prioritization, high scores receive low ranks and therefore higher sampling probability:

$$w_i^{\text{score}} \propto \text{rank}_i^{-1/\tau_{\text{temp}}}. \tag{3}$$

Thus the research problem reduces to defining the score function

$$s_\theta(x) \in \mathbb{R}. \tag{4}$$

Replay should then emphasize levels that are not merely hard, but useful for improving the learner.

2 Baseline Regret Scores

The existing PLR baselines score a level using regret-like quantities computed from completed episodes.

2.1 MaxMC

Let $R_n^{\max}$ be the maximum Monte Carlo return observed on level n. The MaxMC score is the episode-time average

$$s_{\text{MaxMC}}(n) = \frac{1}{|\mathcal{T}_n|} \sum_{t \in \mathcal{T}_n} \left(R_n^{\max} - V_\theta(o_{t,n})\right). \tag{5}$$

This estimates how far the current value function is below the best return seen so far. A large score means the level appears to contain unrealized return.

2.2 Positive Value Loss

The positive-value-loss score keeps only positive advantages:

$$s_{\text{PVL}}(n) = \frac{1}{|\mathcal{T}_n|} \sum_{t \in \mathcal{T}_n} \max(A_{t,n}, 0). \tag{6}$$

This prioritizes levels where the sampled trajectory performed better than the critic expected. It is still a regret proxy: it assumes positive surprise is evidence that the level can teach the agent something.

These scores are sensible baselines, but neither directly asks whether an update on the level improves generalization, policy fit, or held-out loss.

3 Candidate 1: Absolute Policy-Gradient Score

The `abs_pg` direction asks whether a level should be replayed when it induces a large policy update signal. For one timestep and one environment, define the raw policy-gradient loss

$$\ell_{\mathrm{pg}}(\theta; t, n) = -\log \pi_\theta(a_{t,n} \mid o_{t,n})\, A_{t,n}. \tag{7}$$

The per-step score primitive is the Euclidean norm of its parameter gradient:

$$g_{t,n} = \left\| \nabla_\theta \ell_{\mathrm{pg}}(\theta; t, n) \right\|_2 . \tag{8}$$

The level score is the time-average of this quantity over completed episodes:

$$s_{\mathrm{abspg}}(n) = \frac{1}{|\mathcal{T}_n|} \sum_{t \in \mathcal{T}_n} g_{t,n}. \tag{9}$$

Mathematically, this score treats a level as useful when it applies force to the policy parameters. It is attractive because it is closer to the actual actor update than MaxMC or PVL. It is also risky because large gradient norm can mean several different things:

- useful policy error;
- noisy advantage estimates;
- rare high-leverage states;
- instability or scale mismatch in the policy parameterization.

The implementation therefore normalizes advantages before computing the raw gradient norms, and computes the score over rollout timesteps rather than from a single aggregate PPO loss. This gives a matrix $g \in \mathbb{R}^{T \times N}$, which can be reduced by time to obtain one scalar per level.

The open question is whether gradient magnitude is a useful replay signal by itself, or whether it needs to be signed, normalized by update variance, or combined with a held-out progress check.

4 Candidate 2: PPO Value-Loss Score

The `ppo_value_loss` direction isolates critic error. For each timestep and level, define the unclipped value prediction error:

$$\ell_v(t, n) = \frac{1}{2} \left(V_\theta(o_{t,n}) - y_{t,n} \right)^2 . \tag{10}$$

The level score is

$$s_{\mathrm{value}}(n) = \frac{1}{T} \sum_{t=0}^{T-1} \ell_v(t, n). \tag{11}$$

This score prioritizes levels where the critic is poorly calibrated against the GAE target. It is easier to compute than `abs_pg` and is numerically stable, but it only measures value-function fit. It does not directly say that the actor improves after replaying the level.

The hypothesis is that critic uncertainty may still be a strong proxy for useful replay, especially in sparse maze levels where the value function can identify underlearned structure before the policy-gradient signal is clean.

The main mathematical weakness is that value error can be high for reasons that do not help the actor: target noise, bootstrapping error, or stochasticity in episode termination. This score is therefore best viewed as a cheaper baseline against the more direct learning-progress estimator.

5 Candidate 3: Normalized Holdout Learning Progress, S_{in}

The `s_in` direction is the closest to the research goal. Instead of asking whether a level is hard, surprising, or gradient-producing, it asks whether a small virtual update on one rollout improves PPO loss on an independent rollout from the same level.

For each level x_n, collect two independent rollouts:

$$D_A(n), D_B(n) \sim \pi_\theta \text{ interacting with } x_n. \tag{12}$$

D_A is the update batch. D_B is the holdout evaluation batch. Starting from the current training state θ, run K virtual PPO updates on D_A:

$$\theta'_n = U_K(\theta, D_A(n)). \tag{13}$$

Then evaluate the PPO total loss on D_B before and after the virtual update. The implementation uses the per-level PPO loss

$$L_{\text{PPO}}(\theta; D_B) = L_{\text{clip}}(\theta; D_B) + c_v L_{\text{vf}}(\theta; D_B) - c_H H(\theta; D_B). \tag{14}$$

This uses the clipped policy objective, clipped value loss, and entropy bonus matching the normal PPO training objective.

The normalized learning-progress score is

$$S_{\text{in}}(n) = \frac{L_{\text{PPO}}(\theta; D_B(n)) - L_{\text{PPO}}(\theta'_n; D_B(n))}{L_{\text{PPO}}(\theta; D_B(n)) + \epsilon}. \tag{15}$$

A positive value means that virtual training on D_A reduced held-out loss on D_B. A value near zero means little measurable progress. A negative value means the virtual update harmed held-out fit.

This score is important because it separates three concepts that earlier signals conflate:

- difficulty: the level currently has high loss or regret;
- update magnitude: the level produces a large gradient;
- learnability: an update on the level improves independent data from that level.

S_{in} is therefore the most faithful current estimator of "useful learning." Its cost is much higher: each scored level requires independent A/B rollouts, virtual optimization, and held-out PPO evaluation.

6 Current Experimental Position

The project is now positioned to compare replay signals under the same robust PLR training loop. The relevant score functions are:

Score function	Description
`MaxMC`	baseline regret against best observed return
`pvl`	baseline positive advantage / positive value loss
`abs_pg`	raw actor-gradient magnitude
`ppo_value_loss`	critic target error
`s_in`	held-out PPO loss reduction after virtual updates

The default experimental setting in the current run scripts is robust PLR on maze levels with:

Parameter	Value
`num_train_envs`	32
`num_steps`	256
`num_updates`	30000
`replay_prob`	0.8
`level_buffer_capacity`	4000
`prioritization`	rank
`staleness_coeff`	0.3
`n_walls`	60

Evaluation is against the fixed maze suite:

`SixteenRooms`, `SixteenRooms2`, `Labyrinth`, `LabyrinthFlipped`,
`Labyrinth2`, `StandardMaze`, `StandardMaze2`, `StandardMaze3`.

So the journey has moved from "implement standard UED algorithms" to "use PLR as an experimental harness for learning-progress-aware level selection."

7 What Is Already Mathematically Settled

The following pieces are now explicit enough to treat as fixed for the next round of experiments:

1. The PLR buffer mechanics are not the research variable. They provide the replay substrate.
2. `MaxMC` and `pvl` are the regret-style controls.
3. `abs_pg` defines actor-side update magnitude at the per-timestep level.
4. `ppo_value_loss` defines critic-side prediction error as a per-level score.
5. `s_in` defines a direct holdout learning-progress estimator.
6. All candidate scores reduce to one scalar per level, allowing the same sampler to rank, insert, replay, and update levels.

This is a useful point in the journey because the score definitions now share a common interface while expressing different theories of what replay should mean.

8 What Remains Open

The research questions are now empirical and interpretive rather than merely implementational.

8.1 Does high gradient norm predict improvement?

`abs_pg` may identify levels with strong policy signal, but it may also prioritize noise. A useful diagnostic is the correlation between $s_{\text{abspg}}(n)$ and subsequent improvement in evaluation return or $S_{\text{in}}(n)$.

8.2 Is critic error enough?

`ppo_value_loss` is cheap and stable. The question is whether critic error is aligned with useful policy learning. If it performs close to S_{in}, then the expensive virtual-update estimator may not be necessary.

8.3 Does S_{in} justify its compute cost?

S_{in} is conceptually strongest but computationally expensive. Its value depends on whether it produces better level curricula than cheaper proxies. The key comparison is not only final return, but sample efficiency and replay composition over training.

8.4 Should negative learning progress be retained?

The current formula permits negative scores:

$$S_{\text{in}}(n) < 0 \tag{16}$$

when a virtual update worsens held-out PPO loss. This is mathematically meaningful, but the replay sampler ranks scores. We need to decide whether negative progress should suppress replay, be clipped to zero, or be treated as a warning signal for overly hard or unstable levels.

8.5 Should scores be normalized across training time?

The scale of gradients, value errors, and PPO losses changes throughout training. Rank prioritization reduces some scale sensitivity, but cross-time comparisons still matter for buffer insertion and replacement. A natural next step is to track score distributions over time and consider running normalization or quantile-based insertion rules.

9 Near-Term Research Plan

The next mathematical milestone is to turn the score definitions into a clean comparison table:

Score function	What it estimates	Expected failure mode
`MaxMC`	unrealized return	stale best-return memory
`pvl`	positive surprise	advantage noise
`abs_pg`	actor update magnitude	noisy/high-variance gradients
`ppo_value_loss`	critic prediction error	value-only misalignment
`s_in`	held-out learning progress	high compute cost

The next experimental milestone is to run controlled robust-PLR sweeps where only the score function changes. The primary comparisons should be:

1. final evaluation return on the fixed maze suite;
2. learning speed as a function of environment steps;
3. buffer composition over time;
4. score distribution over time;

5. correlation between cheap proxies and S_{in}.

If `abs_pg` or `ppo_value_loss` correlates strongly with S_{in}, then the project can move toward a cheaper approximation of learning-progress replay. If not, then S_{in} becomes the principled target, and the next problem is to make it cheaper through subsampling, fewer virtual updates, cached rollouts, or less frequent scoring.

10 Current Thesis

The working thesis is:

> Regret-based PLR replays levels that appear hard, but the better replay rule is to prioritize levels whose data induces measurable learning progress under the current optimizer.

The present codebase has reached the point where this thesis is testable. The mathematical objects are defined, the score functions share the same replay interface, and the remaining work is to determine which score is predictive enough to guide curriculum formation in practice.